CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler 0x önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Ali Paşaoğlu
Öğrenci numarası (SID)	5001230022 (32 bite sığmadığı için LSB'den MSB'ye yerleştirilen alt 32 bit: 706262726 = 0x2A18B6C6)
E-posta	alipasaoglu@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G3 — Mesaj şifreleme ve bütünlük
Seçilen ISA ve sürümü (ör. RV32IM)	RISC-V RV32IM, ABI ilp32
Derleyici ve sürümü	riscv64-unknown-elf-gcc 8.3.0 (SiFive freedom-tools 2020.04.1, Windows x86-64); -march=rv32im -mabi=ilp32 -O0
Simülatör / araçlar	Ripes v2.3.1 (Single-cycle processor, RV32IM); Visual Studio (ön doğrulama)
Git deposu adresi	https://github.com/alielric/CMPE-3209-Computer-Organization

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	500123002[2]	2
D2	SID'in son iki hanesi	50012300[22]	22
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+2+2	15

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
N	24 + (D2 mod 25)	24 + (22 mod 25) = 24 + 22	46 eleman (184 byte)
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 2 satırı	256 byte / 16 byte / direct / 16 line / set yok
Write policy	d çift → write-back, d tek → write-through	d = 2 çift	write-back
k	(S mod 7) + 3	(15 mod 7) + 3 = 1 + 3	4
Interrupt türü	S çift → Timer, S tek → I/O	S = 15 tek	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 2 = 1.2; τ = 1 / 1.2 GHz	1.2 GHz, τ ≈ 0.833 ns (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T2 = 20 + 5 × (2 mod 4) = 20 + 10	T1 = 1 cycle, T2 = 30 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (15 mod 7) = 2 + 1	3 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	K = SID mod 256 = 706262726 mod 256 (0x2A18B6C6'nın en düşük byte'ı)	K = 198 = 0xC6 (0 değil, 0xA5 gerekmez)

Not: Numara (5001230022) ikilik tabanda 33 bittir ve 32 bite sığmaz. Duyurudaki kurala göre LSB'den MSB'ye doğru yerleştirilerek alt 32 biti kullanılmıştır: 5001230022 − 2^{32} = 706262726 = 0x2A18B6C6. K bu değerden hesaplanmıştır. d, D2 ve S yönergedeki örnek hesapta olduğu gibi numaranın onluk hanelerinden hesaplanmıştır.

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	4	line = 16 byte = 2^4 → w = 4
Line (r) veya set (d)	4 (line, r)	direct mapping, 16 line = 2^4 → r = 4
Tag (s − r veya s − d)	24	s = 32 − w = 28; s − r = 28 − 4 = 24
Toplam	32	24 + 4 + 4 = 32 bit (RV32 adresi)

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☒ **G3 Mesaj şifreleme ve bütünlük**
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> *Girdi dizisi, yönergedeki LCG üretecini çalıştıran lcg_next() fonksiyonuyla SID'den (x_0 = 706262726) üretilir ve 46 elemanlı global msg[] dizisine 'A'–'Z' aralığında yazılır. Dizi K = 198 (0xC6) anahtarıyla yerinde XOR'lanarak şifrelenir; şifreli dizinin checksum'ı, 32-bit sola döndürmeyi yapan rotl1() fonksiyonuyla $c \leftarrow rotl(c, 1) + v_i$ biçiminde hesaplanıp checksum değişkenine yazılır. Ardından dizi aynı anahtarla tekrar XOR'lanarak çözülür ($(x \oplus K) \oplus K = x$). Karşılaştırma için orijinalin kopyası tutulmaz; üreteç SID'den yeniden çalıştırılır ve sonuç decrypt_ok değişkenine yazılır. Böylece bellekte tek bir dizi bulunur ve cache izi yalnızca msg[] ile stack erişimlerinden oluşur.*

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
1  /* Ali Pasaoglu, 5001230022, G3 Mesaj sifreleme ve butunluk */
2  #define N 46                    /* N = 24 + (22 mod 25) */
3
4  int msg[N];                     /* mesaj dizisi ('A'-'Z') */
5  unsigned int checksum;          /* sonuc 1 */
6  int decrypt_ok;                 /* sonuc 2 */
7
8  unsigned int lcg_next(unsigned int x) {
9      return 1664525u * x + 1013904223u;
10 }
11
12 unsigned int rotl1(unsigned int c) {
13     return (c << 1) | (c >> 31);
14 }
15
16 int main(void) {
17     const int SID = 706262726;      /* 5001230022 alt 32 bit: 0x2A18B6C6 */
18     int key = SID % 256;            /* K = 198 = 0xC6 */
19     if (key == 0)
20         key = 0xA5;
21     unsigned int x = SID;
22     int i;
23     for (i = 0; i < N; i++) {       /* 1. gecis: girdi uret */
24         x = lcg_next(x);
25         msg[i] = (int)((x >> 16) % 26) + 65;
26     }
27     for (i = 0; i < N; i++)         /* 2. gecis: sifrele */
28         msg[i] = msg[i] ^ key;
29     unsigned int c = 0;
30     for (i = 0; i < N; i++)         /* 3. gecis: checksum */
31         c = rotl1(c) + (unsigned int)msg[i];
32     checksum = c;
33     for (i = 0; i < N; i++)         /* 4. gecis: sifreyi coz */
34         msg[i] = msg[i] ^ key;
35     x = SID;                        /* ureteci bastan calistir */
36     decrypt_ok = 1;
37     for (i = 0; i < N; i++) {       /* 5. gecis: orijinalle karsilastir */
38         x = lcg_next(x);
```

```
39          if (msg[i] != (int)((x >> 16) % 26) + 65)
40              decrypt_ok = 0;
41      }
42      return 0;
43  }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	4; 23–41	46 elemanlı global int msg[] dizisi üzerinde beş ayrı for döngüsüyle geçiş yapılır.
Y2	Koşullu dallanma	19, 39	Anahtarın 0 olup olmadığı ve çözülen elemanın orijinalle aynı olup olmadığı if ile denetlenir.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	8–10, 12–14; çağrı 24, 31, 38	lcg_next(x) ve rotl1(c) unsigned parametre alıp değer döndürür ve jal ile çağrılır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	9, 13, 25, 28	LCG'de çarpma ve toplama, rotl1'de shift ve OR, harf üretiminde shift ve mod, şifrelemede XOR yapılır.
Y5	Dizi elemanına store	25, 28, 34	msg[i]'ye üretilen, şifrelenen ve çözülen değer yazılır (sw).
Y6	const int SID ve hesapta kullanımı	17; kullanım 18, 21, 35	const int SID = 706262726 hem K = SID mod 256 anahtarını hem de üretecin başlangıç değerini belirler.
Y7	Sonucun tutulduğu global değişken(ler)	5–6; atama 32, 36, 40	Sonuçlar global checksum ve decrypt_ok değişkenlerinde tutulur.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
checksum	0x00011D0C	0x255D2CDD (626 863 325)	mem_checksum.png
decrypt_ok	0x00011C50	0x00000001 (1, çözme başarılı)	mem_decrypt_ok.png
msg[0] (çözülmüş)	0x00011C54	0x00000054 ('T')	mem_decrypt_ok.png
msg[3] (kernel elemanı)	0x00011C60	0x0000004C ('L')	mem_decrypt_ok.png

Adresler gp = 0x00012000 (_start: auipc x3 0x2 + addi x3 x3 −144) üzerinden bulunmuştur: decrypt_ok = gp − 944, msg = gp − 940, checksum = gp − 756. Çözülen mesaj: TVBLGEPYHUHVASUMHGXGFNZYUJBCLDJKQYYNYUPMKNQCNI.

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	9 (lcg_next dönüşü) ve 23–25 (1. geçiş: msg[i] yazma, i++ ve döngü koşulu)
Başlangıç ve bitiş adresi (hex)	0x0001017C (jalr x0 x1 0 = ret) → 0x00010210 … 0x00010250 (bge x15 x14 −72)
Dinamik instruction sayısı	18 (1 + 17): load ×4, store ×3, ALU ×8, remu ×1, koşullu branch ×1, return ×1
Hangi döngü iterasyonu (ör. i = 3)	i = 3 (msg[3] = 0x00011C60, değer 0x4C = 'L')

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.txt, C satırlarıyla açıklamalı hali asm_O0_eslesme.txt). Adresler 0x0001xxxx biçimindedir; tablo lcg_next, rotl1 ve main'deki 162 instruction'ın tamamını kapsar.

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
8–10	lcg_next(x) gövdesi: return 1664525u*x + 1013904223u	4 instr.: prolog: stack çerçevesi, x'i stack'e yaz 8 instr.: lw x; lui+addi 1664525; mul; lui+addi 1013904223; add; a0 = sonuç 3 instr.: epilog + ret	0x10144–0x10150 0x10154–0x10170 0x10174–0x1017C
12–14	rotl1(c) gövdesi: (c << 1) | (c >> 31)	4 instr.: prolog 5 instr.: lw c; slli 1; srli 31; or; a0 = sonuç 3 instr.: epilog + ret	0x10180–0x1018C 0x10190–0x101A0 0x101A4–0x101AC
16	int main(void) {	4 instr.: prolog: sp −48, ra ve s0 kaydı, s0 = çerçeve	0x101B0–0x101BC
17	const int SID = 706262726;	3 instr.: lui 0x2a18b + addi 1734 → sw −36(s0)	0x101C0–0x101C8
18	int key = SID % 256;	7 instr.: işaretli mod: srai/srli/add/andi/sub → sw −20(s0)	0x101CC–0x101E4
19	if (key == 0)	2 instr.: lw key; bne key, x0, +12	0x101E8–0x101EC
20	key = 0xA5;	2 instr.: addi 165; sw −20(s0)	0x101F0–0x101F4
21	unsigned int x = SID;	2 instr.: lw −36; sw −24(s0)	0x101F8–0x101FC
23	for (i = 0; i < N; i++) — 1. geçiş	2 instr.: i = 0; jal → koşul 3 instr.: i++	0x10200–0x10204 0x1023C–0x10244

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		3 instr.: i ≤ 45 ise bge ile gövdeye dön	0x10248–0x10250
24	x = lcg_next(x);	3 instr.: a0 = x; jal lcg_next; x = a0	0x10208–0x10210
25	msg[i] = (int)((x >> 16) % 26) + 65;	10 instr.: srli 16; remu 26; addi 65; &msg[i] = gp−940 + 4i; sw	0x10214–0x10238
27	for (…) — 2. geçiş (şifrele)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x10254–0x10258 0x1028C–0x10294 0x10298–0x102A0
28	msg[i] = msg[i] ^ key;	12 instr.: lw msg[i]; lw key; xor; sw msg[i]	0x1025C–0x10288
29	unsigned int c = 0;	1 instr.: sw x0, −32(s0)	0x102A4
30	for (…) — 3. geçiş (checksum)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x102A8–0x102AC 0x102D8–0x102E0 0x102E4–0x102EC
31	c = rotl1(c) + (unsigned int)msg[i];	10 instr.: a0 = c; jal rotl1; lw msg[i]; add; sw c	0x102B0–0x102D4
32	checksum = c;	2 instr.: lw c; sw −756(gp)	0x102F0–0x102F4
33	for (…) — 4. geçiş (çöz)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x102F8–0x102FC 0x10330–0x10338 0x1033C–0x10344
34	msg[i] = msg[i] ^ key;	12 instr.: lw; lw; xor; sw	0x10300–0x1032C
35	x = SID;	2 instr.: lw −36; sw −24	0x10348–0x1034C
36	decrypt_ok = 1;	2 instr.: addi 1; sw −944(gp)	0x10350–0x10354
37	for (…) — 5. geçiş (karşılaştır)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x10358–0x1035C 0x1039C–0x103A4 0x103A8–0x103B0
38	x = lcg_next(x);	3 instr.: a0 = x; jal lcg_next; x = a0	0x10360–0x10368
39	if (msg[i] != …)	11 instr.: lw msg[i]; srli; remu; addi 65; beq eşitse +8 atla	0x1036C–0x10394
40	decrypt_ok = 0;	1 instr.: sw x0, −944(gp)	0x10398
42	return 0;	2 instr.: a0 = 0	0x103B4–0x103B8
43	}	4 instr.: epilog: ra/s0 geri yükle, sp +48, ret	0x103BC–0x103C8

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00010074	1912 (0x10074–0x107EB)	Tüm makine kodu: başlangıç kodu (_start, memset, exit vb.) ve programın fonksiyonları: lcg_next 0x10144, rotl1 0x10180, main 0x101B0–0x103CB (toplam 162 instr. = 648 B).
.data / .bss	.data 0x00011800 .sdata 0x00011C28 .bss 0x00011C34	1064 12 220 (0x11C34–0x11D0F)	.data/.sdata: C kütüphanesinin başlangıç verileri. .bss: sıfırla başlatılan global değişkenler; _start'taki memset(gp−972, 0, 220) ile temizlenir: decrypt_ok 0x11C50, msg[46] 0x11C54–0x11D0B, checksum 0x11D0C. gp = 0x00012000.

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

> *Program kodu (.text, 0x00010074–0x000107EB) ile veri (.data 0x00011800 ve msg[] 0x00011C54) aynı 32-bit adres uzayında, yalnızca birkaç KB arayla aynı bellekte durur. İşlemci instruction'ı PC'nin gösterdiği adresten, veriyi lw/sw ile hesaplanan adresten aynı bellek arayüzü üzerinden okur. Örneğin Ripes Memory görünümünde 0x000101C0 adresinde instruction word'ü 0x2A18B7B7 (lui), 0x00011C54 adresinde veri 0x00000054 ('T') görünür.*

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	18 (seçilen pencere) 34 (bir tam iterasyon, lcg_next dahil)	8 (bir tam iterasyon, 0x10198–0x101B4)	-O1'de lcg_next main'in içine alındı (inline): call/ret, prolog ve epilog yok. i sayacı yerine doğrudan adres artırılıyor (x11 += 4; bne x11, x13). Sabitler (1664525, 1013904223, 26) döngüden önce bir kez register'a yükleniyor (0x10184–0x10194).
Load sayısı	4 (pencere) 7 (iterasyon)	0	-O0 x, i ve key değişkenlerini stack'te tutar ve her kullanımda lw ile okur. -O1 hepsini register'da tutar (register allocation): x → x14, &msg[i] → x11.
Store sayısı	3 (pencere) 5 (iterasyon)	1	-O0 her atamadan sonra x ve i'yi stack'e yazar, lcg_next de çerçevesine x8 ve x10'u kaydeder. -O1'de yalnızca msg[i] = … ataması (sw x15, 0(x11)) kalır.
Kullanılan register sayısı	7: x1, x3, x8, x10, x13, x14, x15 (pencere) 8: + x2 (iterasyon)	7: x10, x11, x13, x14, x15, x16, x17	Sayılar yakın ama register'ların rolü farklı. -O0'da x2 ve x8 stack adreslemesi için kullanılır, x14 ve x15 geçici olarak dönüşümlü kullanılır. -O1'de her değer kendi register'ında kalır: x16 = 1664525, x10 = 1013904223, x17 = 26, x13 = dizinin sonu.

Not: Bir iterasyondaki bellek referansı (load + store) -O0'da 12, -O1'de 1'dir; fark tamamen register allocation'dan gelir. -O1'de de SID, lui x14, 0x2A18B + addi x14, x14, 1734 ile yüklenir (0x1017C). K = 198 ise derleyici tarafından derleme

anında hesaplanıp xori x14, x14, 198 komutunun immediate'ine yazılmıştır (sabit katlama). Tüm program için dinamik Ic ve m değerleri Aşama 5a'da simülatörden okunacaktır. -O1 listesi: asm_O1.txt.

2c. Elle machine code (12 instruction; R, I, S, B, U ve J formatlarının hepsi kullanıldı). Kaynak: RISC-V Unprivileged Specification, RV32I/M bölümleri. Her sonuç, Ripes'in gösterdiği machine code ile karşılaştırıldı (mc_dogrulama.png: # 1–6, 8, 9; mc_dogrulama2.png: # 7, 12; mc_dogrulama3.png: # 10, 11; asm_O0.txt). Ayrıca GCC 8.3.0 ile üretilen main_O0.elf'in objdump -d çıktısı (objdump_O0.txt) Ripes listesiyle 478 instruction'ın hepsinde aynıdır; -O1 için de 397/397 (objdump_O1.txt).

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?

1	`0x000101C0`	lui x15, 0x2a18b
		(SID üst 20 bit)
			U	`imm[31:12] [31–12] = 00101010000110001011`
				`rd [11–7] = 01111`
				`opcode [6-0] = 0110111`
					`0x2A18B7B7`	`b7 b7 18 2a`
						`(little-endian)`
							Evet
							Ripes: 2a18b7b7

2	`0x000101C4`	addi x15, x15, 1734
		(SID alt 12 bit)
			I	`imm[11:0] [31–20] = 011011000110`
				`rs1 [19–15] = 01111`
				`funct3 [14–12] = 000`
				`rd [11–7] = 01111`
				`opcode [6-0] = 0010011`
					`0x6C678793`	`93 87 67 6c`
						`(little-endian)`
							Evet
							Ripes: 6c678793

3	`0x000101C8`	sw x15, -36(x8)
		(SID stack'e yazılır)
			S	`imm[11:5] [31–25] = 1111110`
				`rs2 [24–20] = 01111`
				`rs1 [19–15] = 01000`
				`funct3 [14–12] = 010`
				`imm[4:0] [11–7] = 11100`
				`opcode [6-0] = 0100011`
					`0xFCF42E23`	`23 2e f4 fc`
						`(little-endian)`
							Evet
							Ripes: fcf42e23

4	`0x000101CC`	lw x14, -36(x8)
		(load)
			I	`imm[11:0] [31–20] = 111111011100`
				`rs1 [19–15] = 01000`
				`funct3 [14–12] = 010`
				`rd [11–7] = 01110`
				`opcode [6-0] = 0000011`
					`0xFDC42703`	`03 27 c4 fd`
						`(little-endian)`
							Evet
							Ripes: fdc42703

5	`0x000101EC`	bne x15, x0, 12
		(if (key == 0))
			B	`imm[12] [31] = 0`
				`imm[10:5] [30–25] = 000000`
				`rs2 [24–20] = 00000`
				`rs1 [19–15] = 01111`
				`funct3 [14–12] = 001`
				`imm[4:1] [11–8] = 0110`
					`0x00079663`	`63 96 07 00`
						`(little-endian)`
							Evet
							Ripes: 00079663

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				imm[11] [7] = 0 opcode [6–0] = 1100011			
6	0x0001020C	jal x1, -200 <lcg_next> (fonksiyon çağrısı)	J	imm[20] [31] = 1 imm[10:1] [30–21] = 1110011100 imm[11] [20] = 1 imm[19:12] [19–12] = 11111111 rd [11–7] = 00001 opcode [6–0] = 1101111	0xF39FF0EF	ef f0 9f f3 (little-endian)	Evet Ripes: f39ff0ef
7	0x00010160	mul x14, x14, x15 (LCG çarpması (M uzantısı))	R	funct7 [31–25] = 0000001 rs2 [24–20] = 01111 rs1 [19–15] = 01110 funct3 [14–12] = 000 rd [11–7] = 01110 opcode [6–0] = 0110011	0x02F70733	33 07 f7 02 (little-endian)	Evet Ripes: 02f70733
8	0x00010218	srli x14, x15, 16 (x >> 16 (kernel))	I	funct6 (imm[11:6]) [31–26] = 000000 shamt [25–20] = 010000 rs1 [19–15] = 01111 funct3 [14–12] = 101 rd [11–7] = 01110 opcode [6–0] = 0010011	0x0107D713	13 d7 07 01 (little-endian)	Evet Ripes: 0107d713
9	0x00010238	sw x14, 0(x15) (msg[i] = ... (kernel, Y5))	S	imm[11:5] [31–25] = 0000000 rs2 [24–20] = 01110 rs1 [19–15] = 01111 funct3 [14–12] = 010 imm[4:0] [11–7] = 00000 opcode [6–0] = 0100011	0x00E7A023	23 a0 e7 00 (little-endian)	Evet Ripes: 00e7a023
10	0x00010250	bge x15, x14, -72 (döngü koşulu (kernel))	B	imm[12] [31] = 1 imm[10:5] [30–25] = 111101 rs2 [24–20] = 01110 rs1 [19–15] = 01111 funct3 [14–12] = 101	0xFAE7DCE3	e3 dc e7 fa (little-endian)	Evet Ripes: fae7dce3

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?

				`imm[4:1] [11–8] = 1100`
				`imm[11] [7] = 1`
				`opcode [6–0] = 1100011`			

11	`0x00010274`	xor x14, x14, x15
		(msg[i] ^ key)
			R	`funct7 [31–25] = 0000000`
				`rs2 [24–20] = 01111`
				`rs1 [19–15] = 01110`
				`funct3 [14–12] = 100`
				`rd [11–7] = 01110`
				`opcode [6–0] = 0110011`
					`0x00F74733`	`33 47 f7 00`
						`(little-endian)`
							Evet
							Ripes: 00f74733

12	`0x0001017C`	jalr x0, 0(x1)
		(ret (kernel başı))
			I	`imm[11:0] [31–20] = 000000000000`
				`rs1 [19–15] = 00001`
				`funct3 [14–12] = 000`
				`rd [11–7] = 00000`
				`opcode [6–0] = 1100111`
					`0x00008067`	`67 80 00 00`
						`(little-endian)`
							Evet
							Ripes: 00008067

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 1 (lui) ve # 2 (addi)
- ☒ Koşullu branch: # 5 (bne, ileri) ve # 10 (bge, geri)
- ☒ Load: # 4 (lw)
- ☒ Store: # 3 ve # 9 (sw)
- ☒ Fonksiyon çağrısı: # 6 (jal lcg_next); dönüş # 12 (jalr = ret)

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

> *SID = 706262726 = 0x2A18B6C6 sayısının 30 anlamlı biti var. I-type addi'nin immediate'i ise yalnızca 12 bit (−2048 … +2047) olduğundan sabit tek instruction'a sığmaz ve ikiye bölünür: (1) lui x15, 0x2A18B üst 20 biti yükler, alt 12 biti sıfırlar → x15 = 0x2A18B000. (2) addi x15, x15, 0x6C6 (= 1734) alt 12 biti ekler → x15 = 0x2A18B6C6. addi'nin 12-bit immediate'i toplanmadan önce 32 bite işaretle genişletilir (sign extension). Alt 12 bitin en üst biti (bit 11) 1 ise immediate negatif sayılır (değer − 4096); bu farkı kapatmak için lui'ye yazılan üst kısım 1 artırılır: üst = (SID + 0x800) >> 12. Bizim alt 12 bitimiz 0x6C6 = 0110 1100 0110_2; bit 11 = 0 olduğu için immediate pozitiftir ve +1 düzeltmesi gerekmez: (0x2A18B6C6 + 0x800) >> 12 = 0x2A18B. Karşı örnek: alt 12 bit 0x9C6 olsaydı imm = 0x9C6 − 0x1000 = −1594 olur, lui'ye 0x2A18C yazılırdı (0x2A18C000 − 1594 = 0x2A18B9C6).*

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

> *İleri branch (# 5): bne x15, x0, 12, adres 0x000101EC (C 19: key 0 değilse 20. satırı atla). Hedef 0x000101F8'deki lw x15, −36(x8) (C 21). Offset = hedef − PC = 0x101F8 − 0x101EC = 0xC = +12 byte. B-type offset 13 bit işaretli bir sayıdır ve en düşük biti hep 0 olduğundan saklanmaz: +12 = 0000000001100_2 → imm[12] = 0, imm[11] = 0, imm[10:5] = 000000, imm[4:1] = 0110*

> *→ 0x00079663. Geri branch (# 10): bge x15, x14, −72, adres 0x00010250 (1. geçiş döngü koşulu, 45 ≥ i). Hedef 0x00010208 (döngü gövdesinin başı, C 24). Offset = 0x10208 − 0x10250 = −0x48 = −72. 13 bit ikiye tümleyen: +72 = 0 0000 0100 1000$_2$ → tersi + 1 = 1 1111 1011 1000$_2$ (0x1FB8) → imm[12] = 1, imm[11] = 1, imm[10:5] = 111101, imm[4:1] = 1100 → 0xFAE7DCE3. Branch alınırsa yeni PC = PC + offset olur; alınmazsa PC + 4 olur.*

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☒ Pipeline yok, her an tek instruction yürütülür.
- ☒ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☒ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☒ Data bus genişliği = 32 bit (RV32 word).

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

> Ripes'in single-cycle işlemcisi bu modele çok yakındır: her cycle'da tek instruction tamamlanır. Gerçek RISC-V çekirdekleri ise pipeline kullanır ve aynı anda 5 veya daha fazla instruction farklı aşamalarda bulunur; bu yüzden "PC önce → sonra" bir an değil, pipeline boyunca dağılmış bir olaydır. Gerçek donanımda erişimler önce cache'e gider; bus işlemi yalnızca miss'te ve tek word yerine tüm line (burst) için yapılır. Kernel'deki 7 data erişiminin hepsi cache hit olduğundan gerçekte hiç bus işlemi üretmezler. MAR ve MBR programcıya görünmez ve tek bir fiziksel register olarak bulunmayabilir (load/store birimi, ara buffer'lar). Instruction fetch'ler de ayrı bir instruction cache'ten (Harvard tarzı L1) yapılır.

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1	0x1017C → 0x10210	jalr x0 x1 0	0x00008067	iac, if, iod, do (hedef / koşul)	0x1017C	0x00008067	x1 + 0 = 0x10210 (dönüş adresi)	PC = 0x10210 (rd = x0, register yazılmaz)	(0x1017C / 0x00008067 / Memory read)	C
2	0x10210 → 0x10214	sw x10 -24 x8	0xFEA42423	iac, if, iod, oac, os	0x10210 sonra 0x7FFFFFD8	0xFEA42423 sonra 0x297B835A	x8 − 24 = 0x7FFFFFD8 (adres)	— (bellek: M[0x7FFFFFD8] = 0x297B835A)	(0x10210 / 0xFEA42423 / Memory read) (0x7FFFFFD8 / 0x297B835A / Memory write) [A4 cache: hit, set 13]	PM
3	0x10214 → 0x10218	lw x15 -24 x8	0xFE842783	iac, if, iod, oac, of	0x10214 sonra 0x7FFFFFD8	0xFE842783 sonra 0x297B835A	x8 − 24 = 0x7FFFFFD8 (adres)	x15 = 0x297B835A (aynı değer yeniden yazıldı)	(0x10214 / 0xFE842783 / Memory read) (0x7FFFFFD8 / 0x297B835A / Memory read) [A4 cache: hit, set 13]	PM
4	0x10218 → 0x1021C	srli x14 x15 16	0x0107D713	iac, if, iod, do	0x10218	0x0107D713	0x297B835A >> 16 = 0x297B	x14 = 0x0000297B	(0x10218 / 0x0107D713 / Memory read)	DP
5	0x1021C	addi x15 x0 26	0x01A00793	iac, if, iod, do	0x1021C	0x01A00793	0 + 26 = 26	x15 =	(0x1021C / 0x01A00793 /	DP

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
	→ 0x10220							0x0000001A	Memory read)	
6	0x10220 → 0x10224	remu x15 x14 x15	0x02F777B3	iac, if, iod, do	0x10220	0x02F777B3	0x297B mod 26 = 11 (0xB)	x15 = 0x0000000B	(0x10220 / 0x02F777B3 / Memory read)	DP
7	0x10224 → 0x10228	addi x14 x15 65	0x04178713	iac, if, iod, do	0x10224	0x04178713	11 + 65 = 76 (0x4C)	x14 = 0x0000004C	(0x10224 / 0x04178713 / Memory read)	DP
8	0x10228 → 0x1022C	addi x13 x3 -940	0xC5418693	iac, if, iod, do	0x10228	0xC5418693	0x12000 − 940 = 0x11C54	x13 = 0x00011C54 (aynı değer yeniden yazıldı)	(0x10228 / 0xC5418693 / Memory read)	DP
9	0x1022C → 0x10230	lw x15 -28 x8	0xFE442783	iac, if, iod, oac, of	0x1022C sonra 0x7FFFFFD4	0xFE442783 sonra 0x00000003	x8 − 28 = 0x7FFFFFD4 (adres)	x15 = 0x00000003	(0x1022C / 0xFE442783 / Memory read) (0x7FFFFFD4 / 0x00000003 / Memory read) [A4 cache: hit, set 13]	PM
10	0x10230 → 0x10234	slli x15 x15 2	0x00279793	iac, if, iod, do	0x10230	0x00279793	3 << 2 = 12	x15 = 0x0000000C	(0x10230 / 0x00279793 / Memory read)	DP
11	0x10234 → 0x10238	add x15 x13 x15	0x00F687B3	iac, if, iod, do	0x10234	0x00F687B3	0x11C54 + 12 = 0x11C60	x15 = 0x00011C60	(0x10234 / 0x00F687B3 / Memory read)	DP
12	0x10238 → 0x1023C	sw x14 0 x15	0x00E7A023	iac, if, iod, oac, os	0x10238 sonra 0x00011C60	0x00E7A023 sonra 0x0000004C	x15 + 0 = 0x11C60 (adres)	— (bellek: M[0x00011C60] = 0x0000004C)	(0x10238 / 0x00E7A023 / Memory read) (0x00011C60 / 0x0000004C / Memory write) [A4 cache: hit, set 6]	PM
13	0x1023C → 0x10240	lw x15 -28 x8	0xFE442783	iac, if, iod, oac, of	0x1023C sonra 0x7FFFFFD4	0xFE442783 sonra 0x00000003	x8 − 28 = 0x7FFFFFD4 (adres)	x15 = 0x00000003	(0x1023C / 0xFE442783 / Memory read) (0x7FFFFFD4 / 0x00000003 / Memory read) [A4 cache: hit, set 13]	PM
14	0x10240 → 0x10244	addi x15 x15 1	0x00178793	iac, if, iod, do	0x10240	0x00178793	3 + 1 = 4	x15 = 0x00000004	(0x10240 / 0x00178793 / Memory read)	DP
15	0x10244 → 0x10248	sw x15 -28 x8	0xFEF42223	iac, if, iod, oac, os	0x10244 sonra 0x7FFFFFD4	0xFEF42223 sonra 0x00000004	x8 − 28 = 0x7FFFFFD4 (adres)	— (bellek: M[0x7FFFFFD4] = 0x00000004)	(0x10244 / 0xFEF42223 / Memory read) (0x7FFFFFD4 / 0x00000004 / Memory write) [A4 cache: hit, set 13]	PM
16	0x10248 → 0x1024C	lw x14 -28 x8	0xFE442703	iac, if, iod, oac, of	0x10248 sonra 0x7FFFFFD4	0xFE442703 sonra 0x00000004	x8 − 28 = 0x7FFFFFD4 (adres)	x14 = 0x00000004	(0x10248 / 0xFE442703 / Memory read) (0x7FFFFFD4 / 0x00000004 / Memory read) [A4 cache: hit, set 13]	PM
17	0x1024C	addi x15 x0 45	0x02D00793	iac, if, iod, do	0x1024C	0x02D00793	0 + 45 = 45	x15 =	(0x1024C / 0x02D00793 /	DP

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
	→ 0x10250							0x0000002D	Memory read)	
18	0x10250 → 0x10208	bge x15 x14 -72	0xFAE7DCE3	iac, if, iod, do (hedef / koşul)	0x10250	0xFAE7DCE3	45 ≥ 4 → dallan; PC + (−72)	PC = 0x10208 (dallanma alındı)	(0x10250 / 0xFAE7DCE3 / Memory read)	C

Kategoriler: lw/sw processor–memory (PM, 7 instr.), srli/addi/remu/slli/add data processing (DP, 9 instr.), jalr/bge control (C, 2 instr.); kernel'de I/O olmadığı için processor–I/O (PIO) yok. PC'nin sıra dışı değişimi: # 1 jalr x0, 0(x1) PC'yi x1'deki dönüş adresine (0x10210) atar; # 18 bge alınınca PC = 0x10250 + (−72) = 0x10208 olur ve döngü başa döner. Diğer 16 instruction'da PC = PC + 4. Kernel'in yürütülmesi i = 3 iterasyonudur (simülatör: rvsim.py; değerler Ripes ile aynı programdan).

Kernel toplamları	Sayı
Instruction fetch	18
Operand fetch (of)	4 (lw: # 3, 9, 13, 16)
Operand store (os)	3 (sw: # 2, 12, 15)
PC'nin sıra dışı değiştiği instruction	2 (# 1 jalr, # 18 bge)

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1a	Fetch: IR ← M[PC], PC ← PC + 1	300 → 301	`3005`	—	—
1b	Execute: AC ← I/O(5): t = (x >> 16) mod 26 = 000B	301	`3005`	000B	—
2a	Fetch: IR ← M[PC], PC ← PC + 1	301 → 302	`2940`	000B	—
2b	Execute: M[940] ← AC (t geçici hücreye)	302	`2940`	000B	M[940] = 000B
3a	Fetch: IR ← M[PC], PC ← PC + 1	302 → 303	`1941`	000B	—
3b	Execute: AC ← M[941] = 0041 (65)	303	`1941`	0041	—
4a	Fetch: IR ← M[PC], PC ← PC + 1	303 → 304	`5940`	0041	—
4b	Execute: AC ← AC + M[940] = 0041 + 000B	304	`5940`	004C	—
5a	Fetch: IR ← M[PC], PC ← PC + 1	304 → 305	`2942`	004C	—

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
5b	Execute: M[942] ← AC: msg[3] = 004C ('L')	305	`2942`	004C	M[942] = 004C
6a	Fetch: IR ← M[PC], PC ← PC + 1	305 → 306	`7006`	004C	—
6b	Execute: I/O(6) ← AC (karakter çıkış birimine)	306	`7006`	004C	I/O(6) = 004C

Program (Fig. 3.4 makinesi: 16-bit word, 4-bit opcode + 12-bit adres; 1 = Load AC, 2 = Store AC, 5 = Add to AC, 3 = Load AC from I/O, 7 = Store AC to I/O): 300: 3005, 301: 2940, 302: 1941, 303: 5940, 304: 2942, 305: 7006. Veri: 940 = t (geçici), 941 = 0041 (sabit 65), 942 = msg[3]. RISC-V'deki srli/remu hesabını bu makine yapamadığı için t, I/O birimi 5'ten okunan bir değer olarak modellendi; değerler kernel'in i = 3 iterasyonundandır (t = 11, msg[3] = 0x4C).

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı	6 (# 7–# 12: addi 65 … sw) srli/remu ile birlikte 9	6 (300–305) 2'si I/O, 1'i geçici store

Tek accumulator'ın getirdiği sınırlama (PDP-11 ADD B,A ile karşılaştırarak):

> Fig. 3.4 makinesinde tek bir accumulator vardır: her işlemde bir operand örtük olarak AC'dedir, diğeri bellektedir ve sonuç yine AC'ye yazılır. Bu yüzden I/O'dan gelen t ile sabit 65'i toplamak için t önce belleğe park edilmek zorundadır (301: Store, 303: Add); fazladan bir instruction cycle ve iki bellek erişimi doğar. PDP-11'in ADD B,A komutu iki bellek operandını tek instruction'da toplar ve sonucu A'ya yazar (memory-to-memory, 8 genel register), yani ara depolamaya gerek kalmaz. RISC-V ise load/store mimarisidir: operandlar 31 genel register'da tutulur (x14 = harf, x15 = adres); belleğe yalnızca lw/sw ile gidilir. -O1'de kernel'in hiç load yapmaması bu çok register'lı yapının sonucudur.

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR	Bellekten okunan / belleğe yazılacak word'ü tutar	Yok; bellek arayüzündeki data buffer (load/store birimi)	Hayır (organization)
MAR	Erişilecek bellek adresini tutar	Yok; adres x8 − 24 gibi ALU'da hesaplanıp bellek arayüzüne verilir	Hayır (organization)
IR	Yürütülen instruction'ın opcode'unu tutar	Dahili instruction register (Ripes'te pipeline/IF çıkışı)	Hayır (organization)
IBR	Word'deki sağdaki ikinci instruction'ı bekletir	Karşılığı yok: her fetch tek 32-bit instruction getirir (en yakını I-cache/prefetch buffer)	Hayır (organization)
PC	Sıradaki instruction çiftinin adresi	pc (RISC-V mimari durumunun parçası; auipc, jal, jalr ve branch'lerle okunur/değişir)	Evet, dolaylı (architecture)
AC	Aritmetik sonuçları ve operandı tutar	Tek accumulator yok; x1–x31 genel register'lar (x0 = 0) bu rolü üstlenir	Evet (architecture: 32 register)
MQ	Çarpımın düşük yarısı / bölümün sonucu	Ayrı register yok; mul düşük 32 biti, mulh yüksek 32 biti, remu kalanı herhangi bir GPR'a	Hayır (işlev GPR'larla,

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
		yazar	architecture)

3c. Interrupt senaryosu (k = 4, tür = I/O; S = 15 tek):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	# 4: 0x00010218 srli x14, x15, 16 (kernel'in 4. instruction'ı).
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	Fig. 3.9: işlemci interrupt'ı yalnızca execute cycle bittikten sonra, bir sonraki fetch'ten önce denetler. srli'nin execute'u (x14 = 0x297B) tamamlanır, ardından interrupt cycle başlar. Instruction atomik olarak biter; yarıda kesilseydi x14 yarım yazılmış olurdu ve dönüşte tekrar mı çalışacağı belirsiz kalırdı (precise interrupt).
Kaydedilen PC değeri (hex)	0x0001021C (k + 1'inci instruction: addi x15, x0, 26) → mepc
Kaydedilen diğer durum bilgisi	mcause = 0x8000000B (bit 31 = 1: interrupt; kod 11 = machine external interrupt, I/O birimi PLIC üzerinden); mstatus.MPIE ← MIE, MIE ← 0, MPP ← önceki ayrıcalık seviyesi; mtval = 0. Genel register'lar (x1–x31) donanımca kaydedilmez; handler kullanacaklarını stack'e yazar.
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	RISC-V M-mode trap: mepc, mcause, mstatus (MIE/MPIE/MPP) CSR'ları; PC ← mtvec.BASE (direct) veya mtvec.BASE + 4 × 11 = BASE + 44 (vectored). Dönüş mret ile: PC ← mepc, MIE ← MPIE.
Kaynak (ISA referansı, bölüm, sayfa)	RISC-V Instruction Set Manual Vol. II: Privileged Architecture, v20211203: §3.1.6.1 mstatus s. 21; §3.1.7 mtvec s. 29; §3.1.14 mepc s. 37; §3.1.15 mcause s. 38 (Tablo 3.6, s. 39); §3.3.2 mret s. 46.
Handler dönüşünde ilk çalışan instruction'ın adresi	0x0001021C (mret, PC'yi mepc'den yükler; addi x15, x0, 26 ile kernel kaldığı yerden devam eder).

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?	Programın kendisi: jal x1, lcg_next (0x1020C) komutu	Dış olay: I/O birimi (asenkron, donanım)
Dönüş adresini kim kaydeder, nereye?	jal donanımı rd = x1 (ra) register'ına 0x10210 yazar; -O0 fonksiyonu gerekirse stack'e kaydeder	Donanım mepc CSR'ına 0x1021C yazar; handler diğer register'ları stack'e kaydeder
User program bundan haberdar mı?	Evet; çağrı kodda açıkça yazılı (Y3)	Hayır; kesinti şeffaftır, program

Ölçüt	Fonksiyon çağrısı	Interrupt
		kaldığı yerden habersiz devam eder

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir	I/O interrupt (öncelik düşük) # 4 srli'nin sonunda tanınır; mepc = 0x1021C, handler A başlar (MIE = 0)	Aynı; handler A mepc/mcause'u stack'e kaydedip MIE = 1 yapar (iç içe kesmeye izin)
t_1: 2. interrupt gelir	Daha yüksek öncelikli interrupt (ör. timer) gelir; MIE = 0 olduğu için beklemede (mip.MTIP = 1)	Hemen tanınır: A durur, handler B başlar (A'nın PC'si yeni mepc)
t_2	A tamamlanır, mret → MIE = 1; bekleyen B hemen tanınır ve B çalışır	B tamamlanır, mret → A kaldığı yerden devam eder
t_3	B tamamlanır, mret	A tamamlanır, kaydettiği mepc = 0x1021C'yi geri yükler, mret
t_4: user program'a dönüş	User program 0x1021C'den devam eder (B, A bitene kadar gecikti)	User program 0x1021C'den devam eder (B gecikmeden servis edildi)

RISC-V varsayılan olarak sequential davranır (trap'te MIE donanımca 0 yapılır); nested yaklaşım yazılımın mepc/mcause'u kaydedip MIE'yi yeniden açmasını gerektirir (Stallings Fig. 3.13a/b).

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)	18	Memory read, Transfer ACK, Clock (adres = PC, data = instruction)
Data read	4	Memory read, Transfer ACK, Clock
Data write	3	Memory write, Transfer ACK, Clock
Toplam	25	—

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek	32 bit; 4 GiB	2^{32} adres × 1 byte = 4 294 967 296 byte = 4 GiB (byte adreslemeli)
Data bus 16 bit olsaydı kernel'in bus cycle sayısı	50 bus cycle	Kernel'deki 25 erişimin hepsi 32-bit (18 fetch + 4 lw + 3 sw); 16-bit data bus'ta her biri 2 transfer: 25

Hesap	Değer	Adımlar
		× 2 = 50
Mevcut / 16-bit oranı	2.0	50 / 25 = 2; aynı kernel 16-bit bus'ta iki kat bus zamanı ister (Problem 3.7 mantığı)

PCIe paket verimi (N × 4 = 46 × 4 = 184 byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)	1	1
Sequence number	2	2
Header	12	16
Data	184	184
ECRC	0	4
LCRC	4	4
STP framing (son)	1	1
Toplam	204	212
Verim = data / toplam	184 / 204 = %90.2	184 / 212 = %86.8
128b/130b ile verim	%90.2 × 128/130 = %88.8	%86.8 × 128/130 = %85.5

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

> (1) Çekirdek → bellek denetleyicisi: veri ana bellekten çekirdeğin çipindeki entegre bellek denetleyicisiyle okunur. (2) Çekirdek/denetleyici → I/O hub: QPI (Fig. 3.17) üzerinden; protokol katmanında packet, link katmanında 80-bit flit, fiziksel katmanda 20-bit phit (20 şerit). (3) I/O hub = PCIe root complex (Fig. 3.21): istek PCIe'ye çevrilir. Transaction layer bir Memory Write TLP oluşturur (header + 184 B data + isteğe bağlı ECRC); data link layer sequence number ve LCRC ekler, ACK/NAK ve akış kontrolü için DLLP'leri kullanır; fiziksel katman STP/END framing ekler ve 128b/130b kodlamayla şeritlere dağıtır. (4) Gerekirse PCIe switch paketi doğru porta yönlendirir; (5) endpoint cihazın data link katmanı LCRC'yi denetleyip ACK DLLP döndürür, transaction layer veriyi cihaz belleğine yazar.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda trace.csv dosyasındadır (-O0, _start'tan exit ecall'una kadar; 2372 referans main ve fonksiyonlarından, 124 referans C kütüphanesinin başlangıç/bitiş kodundan: memset, atexit, exit). Biçim: sira,RW,adres_hex,boyut,c_satiri,stack_mi (E = stack), ek olarak pc ve fonksiyon.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	-O0
Toplam data referansı	2496
Load / store	1612 / 884
Stack erişimi sayısı	2085 (%83.5)
Ana diziye erişim sayısı	368 (msg[], 0x11C54–0x11D0B)

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word
0x00011C3F	0000 0000 0000 0001 0001 1100 | 0011 | 1111	0x00011C	3	15
0x00011C3E	0000 0000 0000 0001 0001 1100 | 0011 | 1110	0x00011C	3	14
0x00011C3D	0000 0000 0000 0001 0001 1100 | 0011 | 1101	0x00011C	3	13
0x00011C3C	0000 0000 0000 0001 0001 1100 | 0011 | 1100	0x00011C	3	12
0x00011C3B	0000 0000 0000 0001 0001 1100 | 0011 | 1011	0x00011C	3	11

w = 4 (16 B line), r = 4 (16 line), s = 32 − 4 = 28, tag = s − r = 24 bit. İzin ilk referansları _start'taki memset'in .bss'i (0x11C34–0x11D0F) sıfırlamasıdır; ilk 12 erişim hizalama için byte (sb) yazmalarıdır.

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1	W	0x00011C3F	0x00011C	3	15	Miss	—	Hayır (ilk kez, compulsory)
2	W	0x00011C3E	0x00011C	3	14	Hit	—	—
3	W	0x00011C3D	0x00011C	3	13	Hit	—	—
4	W	0x00011C3C	0x00011C	3	12	Hit	—	—
5	W	0x00011C3B	0x00011C	3	11	Hit	—	—
6	W	0x00011C3A	0x00011C	3	10	Hit	—	—
7	W	0x00011C39	0x00011C	3	9	Hit	—	—
8	W	0x00011C38	0x00011C	3	8	Hit	—	—
9	W	0x00011C37	0x00011C	3	7	Hit	—	—
10	W	0x00011C36	0x00011C	3	6	Hit	—	—
11	W	0x00011C35	0x00011C	3	5	Hit	—	—
12	W	0x00011C34	0x00011C	3	4	Hit	—	—
13	W	0x00011C40	0x00011C	4	0	Miss	—	Hayır (ilk kez, compulsory)
14	W	0x00011C44	0x00011C	4	4	Hit	—	—
15	W	0x00011C48	0x00011C	4	8	Hit	—	—

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
16	W	0x00011C4C	0x00011C	4	12	Hit	—	—
17	W	0x00011C50	0x00011C	5	0	Miss	—	Hayır (ilk kez, compulsory)
18	W	0x00011C54	0x00011C	5	4	Hit	—	—
19	W	0x00011C58	0x00011C	5	8	Hit	—	—
20	W	0x00011C5C	0x00011C	5	12	Hit	—	—
21	W	0x00011C60	0x00011C	6	0	Miss	—	Hayır (ilk kez, compulsory)
22	W	0x00011C64	0x00011C	6	4	Hit	—	—
23	W	0x00011C68	0x00011C	6	8	Hit	—	—
24	W	0x00011C6C	0x00011C	6	12	Hit	—	—
25	W	0x00011C70	0x00011C	7	0	Miss	—	Hayır (ilk kez, compulsory)
26	W	0x00011C74	0x00011C	7	4	Hit	—	—
27	W	0x00011C78	0x00011C	7	8	Hit	—	—
28	W	0x00011C7C	0x00011C	7	12	Hit	—	—
29	W	0x00011C80	0x00011C	8	0	Miss	—	Hayır (ilk kez, compulsory)
30	W	0x00011C84	0x00011C	8	4	Hit	—	—

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0	— (direct)	0	—	—	— (direct, LRU yok)
1	— (direct)	0	—	—	— (direct, LRU yok)
2	— (direct)	0	—	—	— (direct, LRU yok)
3	— (direct)	1	0x00011C	1	— (direct, LRU yok)
4	— (direct)	1	0x00011C	1	— (direct, LRU yok)
5	— (direct)	1	0x00011C	1	— (direct, LRU yok)
6	— (direct)	1	0x00011C	1	— (direct, LRU yok)
7	— (direct)	1	0x00011C	1	— (direct, LRU yok)
8	— (direct)	1	0x00011C	1	— (direct, LRU yok)
9	—	0	—	—	— (direct, LRU yok)

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
	(direct)				
10	— (direct)	0	—	—	— (direct, LRU yok)
11	— (direct)	0	—	—	— (direct, LRU yok)
12	— (direct)	0	—	—	— (direct, LRU yok)
13	— (direct)	0	—	—	— (direct, LRU yok)
14	— (direct)	0	—	—	— (direct, LRU yok)
15	— (direct)	0	—	—	— (direct, LRU yok)

30. referanstan sonra set 3–8 geçerli ve dirty (memset hep yazar); 6 miss (her yeni line'ın ilk erişimi) ve 24 hit var. Write-back olduğu için bu 30 store henüz hiç bellek yazması üretmedi; write-through olsaydı 30 bellek yazması olurdu. Tüm izde: write-through ile 884 store = 884 bellek yazması; write-back ile yalnızca 89 dirty line yerinden çıkarıldı (+ program sonunda cache'te kalan 11 dirty line: set 0, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14).

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	2496
Hit	24	24	2347 (Ripes: 2347)
Miss	6	6	149 (Ripes: 149)
Hit ratio H	0.8000	0.8000	0.9403 (Ripes: 0.9403)
Ana belleğe yazma sayısı	0	0	89 write-back (+ sonda 11 dirty line) (Ripes: 89)

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

> *Kendi simülatörüm: rvsim.py (RV32IM komut simülatörü; Ripes listesindeki makine kodunu baştan sona çalıştırıp trace.csv'yi üretir) + cachesim.py (direct/set-associative, LRU, write-back + write-allocate). Ayarlar: 256 B, 16 B line, direct, 16 line. Doğrulama: Ripes v2.3.1 data cache (ways 2^0, lines 2^4, words/line 2^2, write-back, write allocate, LRU) aynı programda hit = 2347, miss = 149, writeback = 89 verdi (cache_ripes.png).*

- ☒ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = 16 B)	Hit	Miss	H
Direct	2347	149	0.9403
2-way set-associative	2447	49	0.9804

Mapping (line boyu sabit = 16 B)	Hit	Miss	H
4-way set-associative	2452	44	0.9824
Fully associative (LRU)	2451	45	0.9820

Line boyu (mapping sabit = direct)	Hit	Miss	H
8 B	2358	138	0.9447
16 B	2347	149	0.9403
32 B	2310	186	0.9255
64 B	2153	343	0.8626

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

> Grafik: cache_tasarim_uzayi.png. Direct mapping'de miss'lerin çoğu conflict miss'tir (thrashing): lcg_next/rotl1 çerçeveleri 0x7FFFFFA0–0x7FFFFFBF (set 10–11) ile msg[19..26] = 0x11CA0–0x11CBF aynı set'lere, main'in yerelleri (i, x, key, c: 0x7FFFFFD0–0x7FFFFFDC, set 13) ile msg[31..34] = 0x11CD0–0x11CDF aynı set'e düşer. Bu blokların her geçişte birbirini atması toplam miss'lerin büyük kısmını oluşturur (set 10, 11 ve 13). Set kullanımı dengesizdir: 2496 erişimin 1529'u (%61) set 13'e giderken set 1'e hiç erişilmez (Ripes'teki boş line, cache_ripes.png), set 2–4 de çok az kullanılır. 4g'deki değişiklik bu boş set'leri kullanır. 2-way'de aynı set'e iki blok sığdığı için bu çatışma kalkar ve H 0.940'tan 0.980'e çıkar; 4-way ve fully associative ancak birkaç miss daha azaltır, çünkü kalan miss'ler compulsory'dir (her bloğa ilk erişim). Line boyu 8 B'den 16 B'ye çıkınca H biraz düşer, 32 ve 64 B'de belirgin düşer: stack erişimleri temporal locality'ye dayanır ve aynı birkaç word'e tekrar tekrar gidilir, spatial locality ise yalnızca msg[] taramasında işe yarar. 256 B sabitken line büyüdükçe line sayısı 32 → 4'e iner; stack ve msg aynı az sayıda line için yarışır, büyük line'ların getirdiği komşu word'ler kullanılmadan atılır (cache pollution).

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 = 30	1
Kişiye özel cache	0.9403	1 + 30 × 0.0597 = 2.791	30 / 2.791 = 10.75

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı	— (main.c)	main'e 17. satır eklendi: int pad[36]; (main_v2.c)
Y7 çıktısı (aynı olmalı)	checksum = 0x255D2CDD, decrypt_ok = 1	checksum = 0x255D2CDD, decrypt_ok = 1
H	0.9403	0.9615

	Önce	Sonra
T_s (cycle)	2.791	2.154

Değişikliğin gerekçesi; H artmadıysa nedeni:

> Değişiklik veri yerleşimine yöneliktir. main'e hiç kullanılmayan 144 byte'lık bir yerel dizi eklendi; GCC -O0 bunu main'in stack çerçevesine yerleştirir ve çerçeve 48 → 192 byte büyür. Böylece main'in yerelleri ve lcg_next/rotl1 çerçeveleri 9 line (144 B) aşağı kayar ve msg[]'nin hiç kullanmadığı set 1–4'e düşer; set 10, 11 ve 13'teki thrashing ortadan kalkar. Program aynı işi yapar (Y7 aynı), 2496 referansta miss 149 → 96, write-back 89 → 50. Doğrulama: main_v2.c Ripes'te -O0 ile derlendi (asm_v2_O0.txt), iz rvsim.py ile çıkarıldı (trace_v2.csv); Ripes data cache: hit = 2400, miss = 96 (cache_ripes_v2.png).

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır). Sayılar rvsim.py ile tüm program için (_start → exit) bulundu; Ripes doğrulaması: -O0 çalışması 6478 cycle sürdü (single-cycle işlemcide cycle = instruction; ripes_icache_ic.png), simülatörle aynı. Simülatör main_v2.c için de Ripes ile aynı cache sonucunu verdiğinden -O1 sayıları da aynı yöntemle bulundu. Çarpma sınıfı M uzantısının mul ve remu komutlarını (92 + 92) kapsar; ecall Branch/jump sınıfına alındı.

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1	3167	926
Çarpma	3	184	184
Load	1	1612	220
Store	1	884	228
Branch / jump	2	631	347
Ic (toplam)	Σ I_i (sınıfların toplamı)	**6478**	**1905**
Ortalama CPI	Σ(CPI_i × I_i) / Ic	**7477 / 6478 = 1.154**	**2620 / 1905 = 1.375**

5a-2. Süre ve MIPS rate (f = 1.2 GHz, τ = 0.833 ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	Ic × CPI × τ	6 230.8 ns	2 183.3 ns
MIPS rate	f / (CPI × 10^6)	1039.7	872.5
m (instruction başına data referansı)	data ref. / Ic	2496 / 6478 = 0.3853	448 / 1905 = 0.2352
T, cache yok (k = T2)	Ic × [p + (m × k)] × τ	68 630.8 ns	13 383.3 ns
T, cache var (k = T_s)	Ic × [p + (m × k)] × τ	12 035.8 ns (k = 2.791, H = 0.9403)	3 381.7 ns (k = 3.210, H = 0.9263)

p = ortalama CPI (instruction fetch'lerin 1 cycle'da karşılandığı varsayıldı); k = T2 = 30 cycle (cache yok) veya k = T_s (Aşama 4f; -O1 için aynı cache'te -O1 izi H = 0.9263). T_cache yok / T_cache var oranı: -O0 5.70, -O1 3.96.

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	12 035.8 ns / 3 381.7 ns = 3.56
Speedup cache yok → cache var (-O0)	68 630.8 ns / 12 035.8 ns = 5.70

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

> -O1, -O0'dan 3.56 kat hızlıdır (bellek etkisi olmadan 2.85 kat), ama MIPS rate'i daha düşüktür: 873 < 1040. Sebebi, -O1'in çoğunlukla 1 cycle'lık ALU, load ve store instruction'larını atmasıdır (Ic 6478 → 1905); geriye kalan karışımda 3 cycle'lık mul/remu (184, iki derlemede de aynı) ve 2 cycle'lık branch'lerin payı artar, ortalama CPI 1.154 → 1.375 yükselir. MIPS saniyede kaç instruction yürütüldüğünü ölçer, yapılan işi ölçmez. Farklı derlemeler aynı iş için farklı sayıda instruction ürettiğinden MIPS'e bakan biri yavaş olan -O0'ı daha iyi sanabilir.

5a-3. Amdahl yasası (Nc = 3):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	2. ve 4. geçiş: C 27–28 ve 33–34 (msg[i] = msg[i] ^ key), -O0'da 1666 instruction
Bağımsızlık gerekçesi (veri bağımlılığı)	Her iterasyon yalnızca kendi msg[i]'sini okuyup yazar; key sabittir, bu yüzden iterasyonlar arası bağımlılık yoktur. 1. ve 5. geçişte x_{i+1} = f(x_i) (LCG) zinciri, 3. geçişte c_{i+1} = rotl(c_i) + v_i zinciri iterasyonları sıraya bağlar (loop-carried dependence); bunlar paralelleştirilemez.
f (yürütme süresindeki pay)	f = (1762 + 740 × 2.791) / (7477 + 2496 × 2.791) = 3827 / 14443 = 0.265 (T, cache var, -O0)
Speedup = 1 / [(1 − f) + f / Nc]	1 / [(1 − 0.265) + 0.265 / 3] = 1.215
Nc → ∞ limiti	1 / (1 − 0.265) = 1.361

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1	Instruction formatı (32-bit; R, I, S, B, U, J)	Architecture	Programcının/derleyicinin gördüğü makine kodunu tanımlar	Hayır; ikili kod değişir, yeniden derleme gerekir. -O0 ve -O1 farklı komut dizileriyle aynı checksum'ı verdi.
2	Register sayısı ve genişliği (32 × 32-bit)	Architecture	Programcıya görünür durum	Hayır (yeniden derlemeyle); -O1 farklı register'larla aynı sonucu verdi.
3	Adresleme modları (base + offset, PC-relative)	Architecture	Komutların operand bulma kuralı programın anlamını belirler	Hayır; yalnızca komut sayısı değişir.
4	M uzantısı (mul, remu)	Architecture	Komut kümesinde hangi işlemlerin olduğu	Hayır; rv32i'de mul/remu kütüphane çağrısına döner, Ic artar.

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
5	Interrupt mekanizması (mepc, mcause, mtvec)	Architecture	Ayrıcalıklı yazılımın gördüğü arayüz	Hayır; kesinti şeffaftır, program kaldığı yerden devam eder (3c).
6	Cache boyu ve mapping (256 B, direct)	Organization	Programcıya görünmez, mimariyi gerçekleştirme biçimi	Hayır; 4e'deki tüm konfigürasyonlarda checksum = 0x255D2CDD, ama H 0.940–0.982 arasında, T_s değişti.
7	Line boyu (16 B)	Organization	Programcıya görünmez	Hayır; H 0.863–0.945 değişti, sonuç aynı.
8	Write policy (write-back)	Organization	Programcıya görünmez	Hayır; yalnızca bellek yazma sayısı değişir (89 write-back yerine write-through'da 884).
9	Data bus genişliği (32 bit) ve MAR/MBR	Organization	Programcıya görünmez donanım yapısı	Hayır; 16-bit bus'ta bus cycle sayısı 2 katına çıkar (3d), sonuç aynı.
10	Saat frekansı (1.2 GHz) ve pipeline yokluğu	Organization	Aynı mimarinin farklı gerçekleştirmesi	Hayır; yalnızca T değişir (5a). Ripes'te single-cycle ve 5 aşamalı işlemci aynı programı aynı sonuçla çalıştırır.

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing	0x10220 remu x15, x14, x15	x mod 26 hesabı; ALU'da veri işlenir
Data storage	0x10238 sw x14, 0(x15)	msg[3] = 0x4C bellekte saklanır (0x11C60)
Data movement	0x10214 lw x15, −24(x8)	x bellekten işlemciye taşınır (kernel'de I/O yok; dış dünyaya taşıma 3c'deki I/O olayıdır)
Control	0x10250 bge x15, x14, −72	Döngünün devamına karar verir; PC'yi değiştirir (jalr da control)

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☒ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☒ C kaynak kodu (main.c) ve sonraki 4g sürümü (main_v2.c)
- ☒ asm_O0.txt, asm_O1.txt, asm_v2_O0.txt (Ripes) ve objdump_O0.txt, objdump_O1.txt, objdump_v2_O0.txt (objdump -d); ELF dosyaları: main_O0.elf, main_O1.elf (M2 klasöründe), main_v2_O0.elf
- ☒ trace.csv ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☒ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☒ 4e grafiği
- ☒ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Claude (Anthropic), Claude Opus 5.5	0–5	Adım 0–2: parametre kontrolü, C kodu taslağı, Ripes kurulumu, adres/kernel belirleme, C→assembly eşlemesi, elle encode'un kontrolü. Adım 3–5: RV32IM komut simülatörü ve cache simülatörü yazımı (rvsim.py, cachesim.py), iz ve cache hesapları, 4g değişikliğinin seçimi, performans hesapları, tutanağın doldurulması.	"öğrenci numaram 5001230022, görev olarak G3'ü yapalım"; "araştırma görevlisi mesaj atmış bunu incele sonra da m2 aşamasını yap"; "şimdi kalan adımları yapalım"	Kodu Ripes'te derleyip çalıştırdım; elle encode'ları Ripes'in makine koduyla karşılaştırdım (mc_dogrulama*.png); simülatörün checksum ve decrypt_ok sonucunu Ripes Memory görünümüyle, instruction sayısını Ripes'in cycle sayısıyla ve cache sonuçlarını (main.c ve main_v2.c) Ripes data cache istatistikleriyle karşılaştırdım; hepsi birebir aynı çıktı (cache_ripes.png, cache_ripes_v2.png).

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Ali Paşaoğlu	5001230022		

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		

Bileşen	Azami	Puan	Not
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza